

Tester's Workbench CAST

Tester's Workbench

- This is a complete setup a tester uses to plan, design, execute and manage software testing activities.
- Ensures that all testing is well planned.

Components of a Tester's Workbench

Test plan	What to test, how, when and by whom e.g. test login
Test cases/ Test scripts	Step-by- step instructions to test a scenario e.g. verify user can log in successfully
Test data	Valid and invalid data used during testing e.g. sample user accounts
Test environment	Setup where testing takes place e.g. PC, FMS application
Testing tools	Tools that assist testing activities
Defect tracking system	Recording and managing bugs
Test logs and reports	Records of passed/failed tests and defects found

CAST

- Computer Aided Software Testing (CAST) refers to the use of software tools to support and automate different activities in the software testing process.
- CAST makes testing faster, easier and more accurate.

Types of CAST Tools

Test Management Tools	Help plan, organize and track testing activities Used for preparing test plans, test cases and reports	Documentation: MS Word, Google Docs, Confluence Test planning and tracking: TestRail, HP ALM Reporting: Zephr, MS Excel, Power BI
Test Execution (Automation) Tools	Used to automatically run test cases Used for regression and repetitive tests	Selenium, JUnit, TestProject, Katalon Studio
Performance Testing Tools	Check speed, load and stability	Apache JMeter, LoadRunner
Static Testing Tools	Analyze code without running it to detect code smells , security issues, syntax errors, code standards violations	SonarQube, Checkstyle, FindBugs, PMD
Defect Tracking Tools	Used to record and manage bugs	Jira, Bugzilla, MantisBT
Test Data Generation Tools	Generate data for testing when large data sets are required or real data is unavailable	Mockaroo, Faker, Generatedata.com

CASTTools

Advantages	Disadvantages
Faster execution of tests	High initial cost
Higher test coverage	Requires skilled personnel
Reusable test scripts	Not all tests can be automated e.g. usability testing
Reduces human errors	
Better reporting and tracking	

Testing Applications

- Testing different types of systems depends on the architecture and usage environment.
- **Web applications:** These run in browsers and depend on networks, servers and user interactions.
 - Tools: Selenium, Postman
- **Object-Oriented Applications:** These systems are built using classes and objects.
 - Tools: JUnit, NUnit, pytest, Selenium, Katalon Studio, IDEs
- **Mobile Applications:** Run on phones.
 - Tool: Android Studio Emulator

Testing Web Applications

Functional testing	Check login, registration, forms, transactions	Input: Fees = 50,000 Expected: Income saved and balance increases Bug: Payment saves but income balance not updated
Usability testing	Is the interface easy to use? Check navigation, layout, readability	Can a new student pay fees without help? Issue: Labels not clear
Compatibility testing	Check if the system works on different browsers and devices	Browsers (Chrome, Edge,...) Devices (laptop, phone,...) Issue: Login Button misaligned in Firefox

Testing Web Applications

Performance testing	Check page loading time, response time under many users	100 bursars logging in at once Expected: System should not crash, Pages load quickly
Security testing	Check if system prevents unauthorized access, protects user data	Enter username: 'OR'1'='1 'admin-- Expected: Login fails
Database testing	Verify data is saved correctly, retrieval works and no duplicates or loss	Student pays 20,000 Expected: Database updates balance, transaction is recorded

Why Web Apps run on Computers Vs Mobile phones

Factor	Computer	Mobile
Screen size	Large, detailed view	Small, limited space
Data entry	Fast input on keyboard	Slow touch typing
Performance	High processing power	Limited processing power for heavy tasks
Reports/analysis	Easy to view complex tables, graphs	Hard to display
Security control	Easier to control access in office	Harder to restrict usage
Design	Designed for complex systems	Best for simple tasks

Testing OO Applications

Unit testing	Test individual classes and methods	<code>calculateBalance()</code>
Integration testing	Test interaction between objects/ classes	Account class: <code>stores income balance</code> Transaction class: <code>stores amount</code> Account class uses transactions to update balance
Inheritance testing	Check if child classes correctly inherit properties and methods from parent classes	Parent class: <code>User</code> Methods: <code>login(), logout()</code> Child class: <code>Student pays fees</code> Create student account and Test <code>login()</code> , Test <code>logout()</code>
Polymorphism testing	Verify same method name behaves differently depending on the object	Method: <code>calculateFees()</code> Object1: Student(Day scholar) pays tuition only Object2: Student(Boarding) pays tuition and hostel fee
Encapsulation testing	Ensure data is hidden and accessed only through methods	Student account: <code>feesBalance</code> (private) Accessed through: <code>getBalance()</code>

Testing Mobile Applications

Interrupt testing	Check that system resumes normally after an interrupt	While checking fees balance incoming call arrives Expected: App pauses and resumes Issue: App crashes after call ends
Usability testing	Is the interface easy to use? Check navigation, layout, readability	Parent views fees balance Expected: Balance displays clearly Issue: Text too small to read on small phone
Network testing	Check response after switching between networks Verify system works offline where applicable	Switch from Wi-Fi to Mobile data Expected: App still loads data Issue: App shows blank screen offline

Testing Desktop Applications

Installation testing	Check that system installs correctly	Install software e.g. FMS, MS Office Expected: Installs without errors Issue: Missing shortcut
Performance testing	Check system response time	Load 10,000 student records Expected: System responds within seconds Issue: System freezes due to memory overload
Compatibility testing	Does system run on different platforms correctly	Test on Windows 10, 11, MacOS, Linux Expected: Same behaviour Issue: System requires framework on Windows 10



END